Generic Programming: The Good, the Bad, and the ...

Guntram Berti
Consulting Mathematical Methods
Bonn

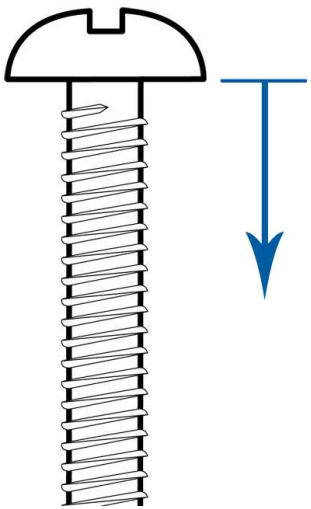
28. February 2015

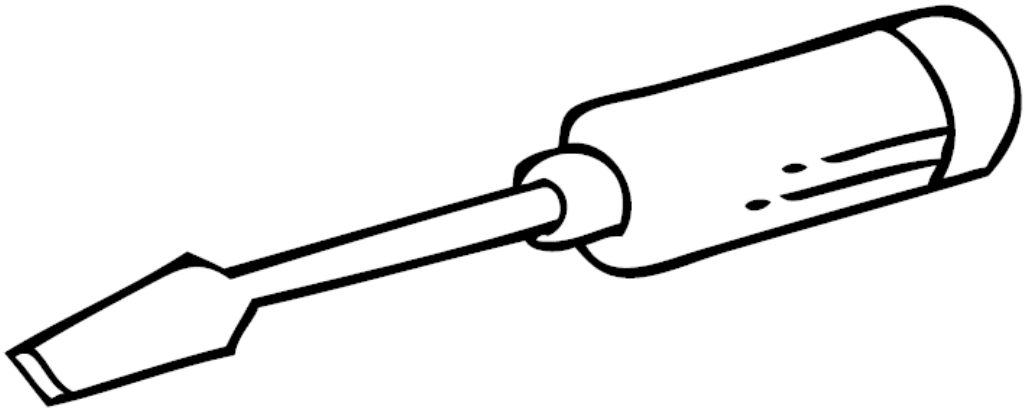
Contents

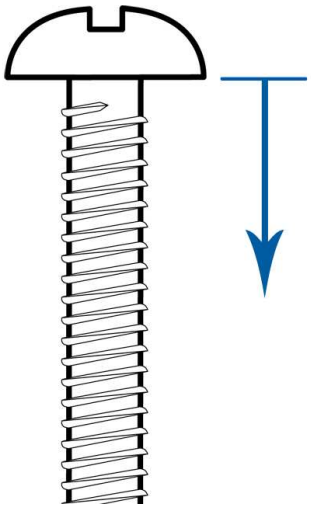
Contents
Motivation
Lifting
Problems
No End
Internal Structure

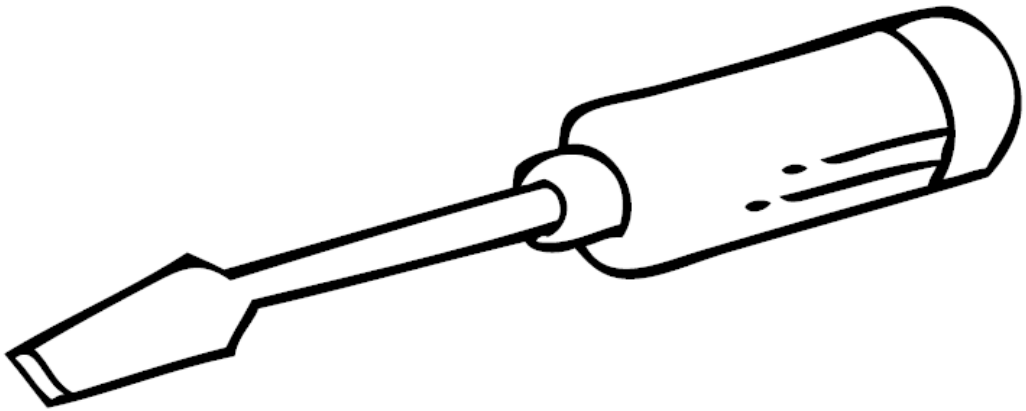
Маршрут

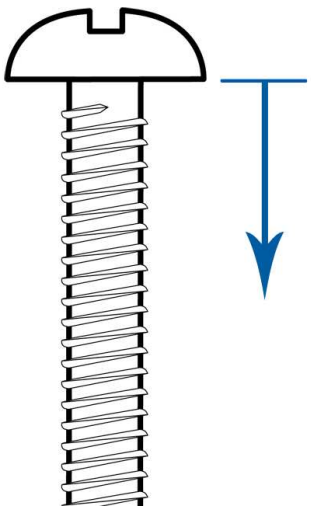
- A Problem: Reusing (algorithmic) code
- Lifting a simple sum
- Problems with sum
- Fixing the problems
- Discussion



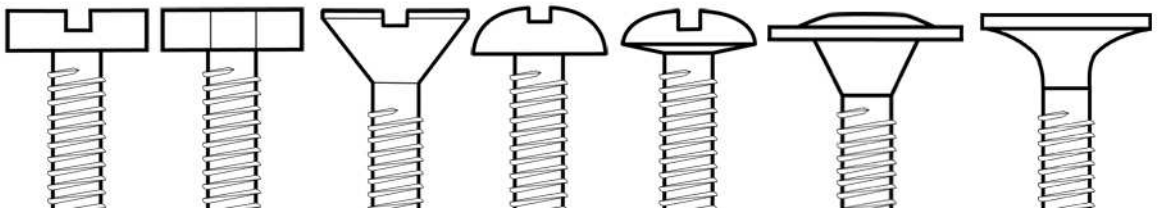


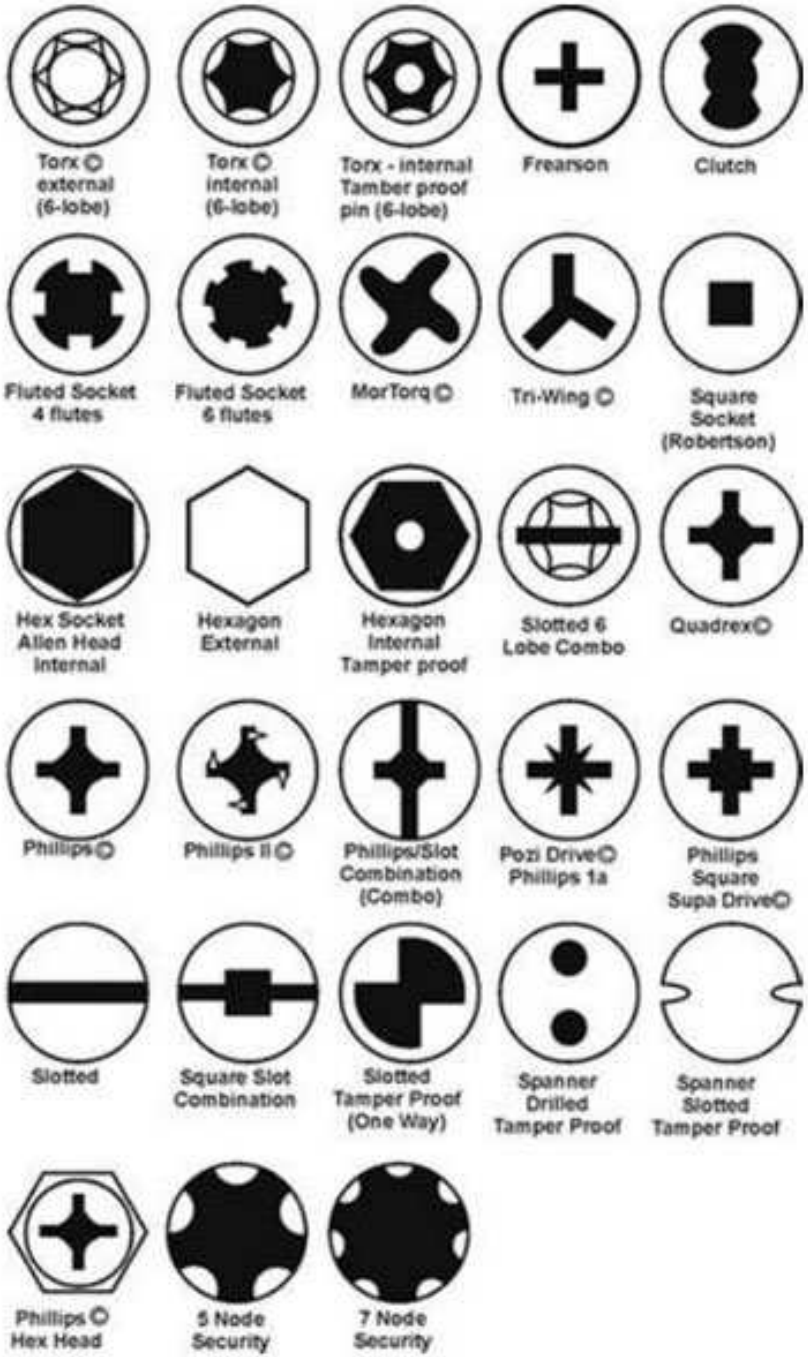






✔




Torx © external (6-lobe)
Torx © internal (6-lobe)
Torx - internal Tamber proof pin (6-lobe)
Frearson
Clutch
Fluted Socket 4 flutes
Fluted Socket 6 flutes
MorTorq ©
Tri-Wing ©
Square Socket (Robertson)
Hex Socket Allen Head Internal
Hexagon External
Hexagon Internal Tamper proof
Slotted 6 Lobe Combo
Quadrex©
Phillips©
Phillips II ©
Phillips/Slot Combination (Combo)
Pozi Drive© Phillips 1a
Phillips Square Supa Drive©
Slotted
Square Slot Combination
Slotted Tamper Proof (One Way)
Spanner Drilled Tamper Proof
Spanner Slotted Tamper Proof
Phillips © Hex Head
5 Node Security
7 Node Security



8240



8242



8243



8244



8245



8440




Does not scale!

Is software any different?

Reusing algorithmic code: A problem?

Contents
Motivation
traditional sum
reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

Reusing algorithmic code: A problem?

Contents
Motivation
traditional sum
reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

Traditional implementation (of a simple algorithm):

Reusing algorithmic code: A problem?

Contents
Motivation
traditional sum
reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

Traditional implementation (of a simple algorithm):

```
double sum(double * v, int n) {
  double s = 0;
  for(int i = 0; i < n; ++i)
    s += v[i];
  return s;
}
```

. . . is that (re)usable?

Reusing algorithmic code: A problem?

Contents
Motivation
traditional sum
reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

Traditional implementation (of a simple algorithm):

```
double sum(double * v, int n) {
  double s = 0;
  for(int i = 0; i < n; ++i)
    s += v[i];
  return s;
}
```

. . . is that (re)usable?

```
int * vi;
```

Reusing algorithmic code: A problem?

Contents
Motivation
traditional sum
reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

Traditional implementation (of a simple algorithm):

```
double sum(double * v, int n) {
  double s = 0;
  for(int i = 0; i < n; ++i)
    s += v[i];
  return s;
}
```

. . . is that (re)usable?

```
int * vi;
```

→ `sum` doesn't work . . . `vi`: wrong value type

Reusing algorithmic code: A problem?

Contents
Motivation
traditional sum
reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

Traditional implementation (of a simple algorithm):

```
double sum(double * v, int n) {
  double s = 0;
  for(int i = 0; i < n; ++i)
    s += v[i];
  return s;
}
```

. . . is that (re)usable?

```
int * vi;
```

→ `sum` doesn't work . . . `vi`: wrong value type

```
struct vec3 { double x[3]; ...};   vec3 *v3 = ...;
```

Reusing algorithmic code: A problem?

Contents
Motivation
traditional sum
reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

Traditional implementation (of a simple algorithm):

```
double sum(double * v, int n) {
  double s = 0;
  for(int i = 0; i < n; ++i)
    s += v[i];
  return s;
}
```

. . . is that (re)usable?

```
int * vi;
```

→ `sum` doesn't work . . . `vi`: wrong value type

```
struct vec3 { double x[3]; ...};    vec3 *v3 = ...;
```

→ `sum` doesn't work . . . `v3`: wrong access pattern

Contents
Motivation
traditional sum
reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

`sum` is overspecified

The Problem

Contents
Motivation
traditional sum
reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

`sum` is overspecified
(ridiculously)

Reuse *is* a problem!

Contents
Motivation
traditional sum
reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

`sum` is a simple case . . . there are more complex

- ✖ data structures
- ✖ algorithms
- ✖ hardware (parallel!)

Reuse *is* a problem!

Contents
Motivation
traditional sum
reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

Reuse of algorithmic code *is* difficult.
Traditional approaches have drawbacks:

- ✖ overspecified implementations
- ✖ type informationen missing / hard to get
- ✖ too much encapsulation of data or implementations
- ✖ code replication

Contents
Motivation
traditional sum
reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

Can we do better?


60
55
50
TX45
14
12
10
8
HX7
T 60
T 55
T 50
T 45
HX 14
HX 12
HX 10
HX 8
HX 7
HXL6
7
8
10
12
HX 6
HX 7
HX 8
HX 10
HX 12
HX2
2.5
3
4
5
6
TX8
10
15
20
25
27
30
40
E4
E5
E6
E8
E10
E12
E14
E18

Idea of generic programming

Contents
Motivation
traditional sum
reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

Idea of generic programming

Contents
Motivation
traditional sum reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

Algorithms are as insensitive to changes of data structure as possible.

Alexander Stepanov

Idea of generic programming

Contents
Motivation
traditional sum reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

Algorithms are as insensitive to changes of data structure as possible.

Alexander Stepanov

Goal:
Make implementations as general as possible . . .

Idea of generic programming

Contents
Motivation
traditional sum reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

Algorithms are as insensitive to changes of data structure as possible.

Alexander Stepanov

Goal:
Make implementations as general as possible ...
but not more.

Idea of generic programming

Contents
Motivation
traditional sum reuse problems
generic idea
Lifting
Problems
No End
Internal Structure

Algorithms are as insensitive to changes of data structure as possible.

Alexander Stepanov

Goal:
Make implementations as general as possible ...
but not more.

Process:
Discover & remove artificial restrictions
Translate babylonian babble of data into a unified language

“Lifting” an implementation



What is “lifting”?

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

What is “lifting”?

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

Lifting seeks to discover a generic algorithm by answering the following fundamental question: What are the minimal requirements that my data types need to fulfill for the algorithm to operate correctly and efficiently?

Doug Gregor, generic-programming.org

Finding the assumptions

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

The (implicit) assumptions of `sum`:

```
double sum(double *v, int n) {
  double s = 0.0;
  for(int i = 0; i < n; ++i) {
    s += v[i];
  }
  return s;
}
```

- ✖ Type is `double`
- ✖ Values stored in an array
- ✖ (without gaps)

Finding the assumptions

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

The (implicit) assumptions of `sum`:

```
double sum(double *v, int n) {
  double s = 0.0;
  for(int i = 0; i < n; ++i) {
    s += v[i];
  }
  return s;
}
```

- ✖ Type is `double`
- ✖ Values stored in an array
- ✖ (without gaps)

Finding the assumptions

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

The (implicit) assumptions of `sum`:

```
double sum(double *v, int n) {
  double s = 0.0;
  for(int i = 0; i < n; ++i) {
    s += v[i];
  }
  return s;
}
```

- ✖ Type is `double`
- ✖ Values stored in an array
- ✖ (without gaps)

Finding the assumptions

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

The (implicit) assumptions of `sum`:

```
double sum(double *v, int n) {
  double s = 0.0;
  for(int i = 0; i < n; ++i) {
    s += v[i];
  }
  return s;
}
```

- ✖ Type is `double`
- ✖ Values stored in an array
- ✖ (without gaps)

Finding the assumptions

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

The (implicit) assumptions of `sum`:

```
double sum(double *v, int n) {
  double s = 0.0;
  for(int i = 0; i < n; ++i) {
    s += v[i];
  }
  return s;
}
```

- ✖ Type is `double`
- ✖ Values stored in an array
- ✖ (without gaps)

Generic Sum: Getting rid of type-is-double

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
double sum(double* v, int n) {
  double s = 0.0;
  for(int i = 0; i < n; ++i) {
    s = s + v[i];
  }
  return s;
}
```

Generic Sum: Getting rid of type-is-double

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
double sum(double* v, int n) {
  double s = 0.0;
  for(int i = 0; i < n; ++i) {
    s = s + v[i];
  }
  return s;
}
```

Generic Sum: Getting rid of type-is-double

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class T>
T      sum(T*      v, int n) {
  T       s = 0;
  for(int i = 0; i < n; ++i) {
    s = s + v[i];
  }
  return s;
}
```

Generic Sum: Getting rid of type-is-double

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class T>
T      sum(T*      v, int n) {
  T       s = 0;
  for(int i = 0; i < n; ++i) {
    s = s + v[i];
  }
  return s;
}
```

This was simple. To simple . . . ?

Generic Sum: Getting rid of data-is-in-array . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

Reformulating the code:

```
template<class T>
T sum(T* v, int n) {
  T s = 0;
  for(int i = 0; i < n; ++i) {
    s = s + v[i];

  }
  return s;
}
```

Generic Sum: Getting rid of data-is-in-array . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

Reformulating the code:

```
template<class T>
T sum(T* v, int n) {
  T s = 0;
  for(int i = 0; i < n; ++i) {
    s = s + v[i];

  }
  return s;
}
```

Generic Sum: Getting rid of data-is-in-array . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

Reformulating the code:

```
template<class T>
T sum(T* v, int n) {
  T s = 0;
  for(int i = 0; i < n; ++i) {
    s = s + *v;
    ++v;
  }
  return s;
}
```

Generic Sum: Getting rid of data-is-in-array . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

Reformulating the code:

```
template<class T>
T sum(T* v, int n) {
  T s = 0;
  for(int i = 0; i < n; ++i) {
    s = s + *v;
    ++v;
  }
  return s;
}
```

Generic Sum: Getting rid of data-is-in-array . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

Reformulating the code:

```
template<class T>
T sum(T* v, T* end) { // end == v + n
  T s = 0;
  while(v != end) {
    s = s + *v;
    ++v;
  }
  return s;
}
```

Generic Sum: Getting rid of data-is-in-array . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class T>
T sum(T* v, T* end) {
  T s = 0;
  while (v != end) {
    s = s + *v;
    ++v;
  }
  return s;
}
```

What does `v` have to support?

Generic Sum: Getting rid of data-is-in-array . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class T>
T sum(T* v, T* end) {
  T s = 0;
  while (v != end) {
    s = s + *v;
    ++v;
  }
  return s;
}
```

What does `v` have to support?

- ✖ comparison, `v != end`

Generic Sum: Getting rid of data-is-in-array . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class T>
T sum(T* v, T* end) {
  T s = 0;
  while (v != end) {
    s = s + *v;
    ++v;
  }
  return s;
}
```

What does `v` have to support?

- ✖ comparison, `v != end`
- ✖ increment, `++v` (prefix)

Generic Sum: Getting rid of data-is-in-array . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class T>
T sum(T* v, T* end) {
  T s = 0;
  while (v != end) {
    s = s + *v;
    ++v;
  }
  return s;
}
```

What does `v` have to support?

- ✖ comparison, `v != end`
- ✖ increment, `++v` (prefix)
- ✖ dereference, `*v` (only reading)

Generic Sum: Getting rid of data-is-in-array . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class T>
T sum(T* v, T* end) {
  T s = 0;
  while (v != end) {
    s = s + *v;
    ++v;
  }
  return s;
}
```

What does `v` have to support?

- ✖ comparison, `v != end`
- ✖ increment, `++v` (prefix)
- ✖ dereference, `*v` (only reading)

Concept: Iterator

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

New abstraction: Iterator (generalizing pointer-to-array)

Concept: Iterator

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

New abstraction: Iterator (generalizing pointer-to-array)

```
concept Iterator { // Pseudocode, no C++
  typedef      value_type;
  Iterator&    operator++();
  value_type   operator*();
};
bool operator!=(Iterator const&, Iterator const&);
```

Concept: Iterator

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

New abstraction: Iterator (generalizing pointer-to-array)

```
concept Iterator { // Pseudocode, no C++
  typedef      value_type;
  Iterator&    operator++();
  value_type   operator*();
};
bool operator!=(Iterator const&, Iterator const&);
```

Low requirements, can be broadly supported!

Concept: Iterator

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

New abstraction: Iterator (generalizing pointer-to-array)

```
concept Iterator { // Pseudocode, no C++
  typedef      value_type;
  Iterator&    operator++();
  value_type   operator*();
};
bool operator!=(Iterator const&, Iterator const&);
```

Low requirements, can be broadly supported!

For nit pickers:
`operator*` is called only once at each position (single pass)

Iterators: Example

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

Example: a simply linked list

Iterators: Example

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

Example: a simply linked list

```
struct List { // rudimentary
  int data;
  List* next; // == 0 at end
};
```

Iterators: Example

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

Example: a simply linked list

```
struct List { // rudimentary
  int data;
  List* next; // == 0 at end
};
```

```
struct ListIt { // implements Iterator-Concept
  List* curr;
  int      operator*() const { return curr->data; }
  ListIt& operator++() {
    curr = curr->next; return *this; }
};
```

Iterators: Example

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

Example: a simply linked list

```
struct List { // rudimentary
  int data;
  List* next; // == 0 at end
};
```

```
struct ListIt { // implements Iterator-Concept
  List* curr;
  int      operator*() const { return curr->data; }
  ListIt& operator++() {
    curr = curr->next; return *this; }
};
```

```
inline bool operator!=(ListIt const& a, ListIt const& b)
{ return a.curr != b.curr; }
```

Iterators: Example

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

Example: a simply linked list

```
struct List { // rudimentary
  int data;
  List* next; // == 0 at end
};
```

```
struct ListIt { // implements Iterator-Concept
  List* curr;
  int      operator*() const { return curr->data; }
  ListIt& operator++() {
    curr = curr->next; return *this; }
};
```

```
inline bool operator!=(ListIt const& a, ListIt const& b)
{ return a.curr != b.curr; }
```

ListIt permits read access only

Generic Sum: Using iterators . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class T >
T  sum(T*  v, T*  end) {
  T  s = 0;
  while(v != end) {
    s += *v;
    v++;
  }
  return s;
}
```

Generic Sum: Using iterators . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class Iter>
??? sum(Iter v, Iter end) {
  ??? s = 0;
  while(v != end) {
    s += *v;
    v++;
  }
  return s;
}
```

Generic Sum: Using iterators . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class Iter>
??? sum(Iter v, Iter end) {
  ??? s = 0;
  while(v != end) {
    s += *v;
    v++;
  }
  return s;
}
```

Now, there's a little problem . . .

. . . where is our value type?

Getting back the value type: A solution . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Possible solution: Extra parameter

Getting back the value type: A solution ...

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Possible solution: Extra parameter

```
template<class Iter,class T>
T
sum(Iter v, Iter end, T init) {
  T  s = init;
  while(v != end) {
    s = s + *v;
    v++;
  }
  return s;
}
```

Getting back the value type: A solution ...

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Possible solution: Extra parameter

```
template<class Iter,class T>
T
sum(Iter v, Iter end, T init) {
  T  s = init;
  while(v != end) {
    s = s + *v;
    v++;
  }
  return s;
}
```

+ Full control over value type *and* init value

Getting back the value type: A solution . . .

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Possible solution: Extra parameter

```
template<class Iter,class T>
T
sum(Iter v, Iter end, T init) {
  T  s = init;
  while(v != end) {
    s = s + *v;
    v++;
  }
  return s;
}
```

\+ Full control over value type *and* init value

— mandatory extra parameter

Getting back the value type: A solution ...

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Possible solution: Extra parameter

```
template<class Iter,class T>
T
sum(Iter v, Iter end, T init) {
  T  s = init;
  while(v != end) {
    s = s + *v;
    v++;
  }
  return s;
}
```

+ Full control over value type *and* init value
− mandatory extra parameter
− Is type `T` kown at call site??

Getting back the value type: A solution ...

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Possible solution: Extra parameter

```
template<class Iter,class T>
T
sum(Iter v, Iter end, T init) {
  T  s = init;
  while(v != end) {
    s = s + *v;
    v++;
  }
  return s;
}
```

+ Full control over value type *and* init value
− mandatory extra parameter
− Is type `T` kown at call site??

Another solution maps iterator type to value type `T`

Getting back the value type: Solution 2

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Solution 2: Map `value` : Iterator Type $\mapsto$ Value Type

Getting back the value type: Solution 2

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Solution 2: Map `value` : Iterator Type $\mapsto$ Value Type

General case: `I` $\mapsto$ `I::value_type`

Getting back the value type: Solution 2

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Solution 2: Map `value` : Iterator Type $\mapsto$ Value Type

General case: `I` $\mapsto$ `I::value_type`

```
template<class I> struct value
{ typedef typename I::value_type value_type; };
```

Getting back the value type: Solution 2

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Solution 2: Map `value` : Iterator Type $\mapsto$ Value Type

General case: `I` $\mapsto$ `I::value_type`

```
template<class I> struct value
{ typedef typename I::value_type value_type; };
```

For pointers: `T*` $\mapsto$ `T`

Getting back the value type: Solution 2

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Solution 2: Map `value` : Iterator Type $\mapsto$ Value Type

General case: `I` $\mapsto$ `I::value_type`

```
template<class I> struct value
{ typedef typename I::value_type value_type; };
```

For pointers: `T*` $\mapsto$ `T`

```
template<class T> struct value<T*> {typedef T type;};
```

Getting back the value type: Solution 2

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Solution 2: Map `value` : Iterator Type $\mapsto$ Value Type

General case: `I` $\mapsto$ `I::value_type`

```
template<class I> struct value
{ typedef typename I::value_type value_type; };
```

For pointers: `T*` $\mapsto$ `T`

```
template<class T> struct value<T*> {typedef T type;};
```

Our Iterator: `ListIt` $\mapsto$ `int`

Getting back the value type: Solution 2

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Solution 2: Map `value` : Iterator Type $\mapsto$ Value Type

General case: `I` $\mapsto$ `I::value_type`

```
template<class I> struct value
{ typedef typename I::value_type value_type; };
```

For pointers: `T*` $\mapsto$ `T`

```
template<class T> struct value<T*> {typedef T type;};
```

Our Iterator: `ListIt` $\mapsto$ `int`

```
template<> struct value<ListIt>  {typedef int type;};
```

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Solution 2: Type mapping in action

Getting back the value type

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Solution 2: Type mapping in action

```
template<class Iter>
typename value<Iter>::type
sum(Iter v, Iter end) {
  typename value<Iter>::type s = 0;
  while(v != end) {
    s = s + *v;
    v++;
  }
  return s;
}
```

Getting back the value type

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Solution 2: Type mapping in action

```
template<class Iter>
typename value<Iter>::type
sum(Iter v, Iter end) {
  typename value<Iter>::type s = 0;
  while(v != end) {
    s = s + *v;
    v++;
  }
  return s;
}
```

The standard library implements `value` as `iterator_traits<Iter>::value_type`.

Is sum generic enough?

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

Is sum generic enough?

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
struct employee {
  double salary;
  int    id;
};
vector<employee> staff;
// double salaries = sum(staff.begin(), staff.end());
```

Is sum generic enough?

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
struct employee {
  double salary;
  int    id;
};
vector<employee> staff;
// double salaries = sum(staff.begin(), staff.end());
```

```
vector<string> words = { "This", "is", "a", "Sentence"};
string text = sum(words.begin(), words.end());
// "Thisisasentence"
```

Is sum generic enough?

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
struct employee {
  double salary;
  int    id;
};
vector<employee> staff;
// double salaries = sum(staff.begin(), staff.end());
```

```
vector<string> words = { "This", "is", "a", "Sentence"};
string text = sum(words.begin(), words.end());
// "Thisisasentence"
```

```
double max_salary = ??
```

Is sum generic enough?

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
struct employee {
  double salary;
  int    id;
};
vector<employee> staff;
// double salaries = sum(staff.begin(), staff.end());
```

```
vector<string> words = { "This", "is", "a", "Sentence"};
string text = sum(words.begin(), words.end());
// "Thisisasentence"
```

```
double max_salary = ??
```

No!

Sum gets even more generic ...

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class Iter, class T>
T sum(Iter v, Iter end, T init) {
  T s = init;
  while(v != end) {
    s = s + *v;
    v++;
  }
  return s;
}
```

Sum gets even more generic ...

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class Iter, class T>
T sum(Iter v, Iter end, T init) {
  T s = init;
  while(v != end) {
    s = s + v->salary;
    v++;
  }
  return s;
}
```

Sum gets even more generic ...

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class Iter, class T>
T sum(Iter v, Iter end, T init) {
  T s = init;
  while(v != end) {
    s = s + " " + *v;
    v++;
  }
  return s;
}
```

Sum gets even more generic ...

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class Iter, class T>
T sum(Iter v, Iter end, T init) {
  T s = init;
  while(v != end) {
    s = max(s,*v);
    v++;
  }
  return s;
}
```

Sum gets even more generic ...

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class Iter, class T>
T sum(Iter v, Iter end, T init) {
  T s = init;
  while(v != end) {
    s = ;
    v++;
  }
  return s;
}
```

All having the form `s = op(s,*v)`.

Sum XXL: Reduce

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class Iter, class T>
T sum(Iter v, Iter end, T init) {
  T s = init;
  while(v != end) {
    s = s + *v;
    v++;
  }
  return s;
}
```

Sum XXL: Reduce

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class Iter, class T, class Op>
T reduce(Iter v, Iter end, T init, Op op) {
  T s = init;
  while(v != end) {
    s = op(s,*v);
    v++;
  }
  return s;
}
```

What can we do with reduce?

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

```
vector<employee> staff;
double salaries = reduce(staff.begin(),staff.end(),0.0,
                         [](double s, employee e)
                           {return s + e.salary;} );
```

What can we do with reduce?

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array iterators
value type
Reduce
Problems
No End
Internal Structure

```
vector<employee> staff;
double salaries = reduce(staff.begin(),staff.end(),0.0,
                         [](double s, employee e)
                           {return s + e.salary;} );
```

```
vector<string> w = { "This", "is", "a", "Sentence"};
string text = reduce(w.begin(), w.end(), string(),
                     [](string s, string t)
                       {return s + " " + t;} );
// " This is a sentence" -> note leading space
```

What can we do with reduce?

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

```
vector<employee> staff;
double salaries = reduce(staff.begin(),staff.end(),0.0,
                         [](double s, employee e)
                           {return s + e.salary;} );
```

```
vector<string> w = { "This", "is", "a", "Sentence"};
string text = reduce(w.begin(), w.end(), string(),
                     [](string s, string t)
                       {return s + " " + t;} );
// " This is a sentence" -> note leading space
```

```
double max_salary =
  reduce(staff.begin(), staff.end(),
         staff.begin()->salary,
         [](double s, employee e)
           {return max(s, e.salary);} );
```

Counting with reduce

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class It, class P>
size_t count_if(It begin, It end, P pred);
```

Counting with reduce

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class It, class P>
size_t count_if(It begin, It end, P pred);
```

```
template<class T, class Pred> struct counter {
  size_t operator()(size_t c, T t) const
   { return c + (pred(t) ? 1 : 0); } };
```

Counting with reduce

Contents
Motivation
Lifting
Finding the assumptions
Get rid of double
Get rid of array
iterators
value type
Reduce
Problems
No End
Internal Structure

```
template<class It, class P>
size_t count_if(It begin, It end, P pred);
```

```
template<class T, class Pred> struct counter {
  size_t operator()(size_t c, T t) const
   { return c + (pred(t) ? 1 : 0); } };
```

```
template<class It, class P>
size_t count_if(It begin, It end, P pred)
{
  typedef typename  value<It>::type V;
  return reduce(begin, end, 0,
                counter<V, P>(pred));
}
```

Contents
Motivation
Lifting
Problems
Problems of reduce
No End
Internal Structure

Part II: The Bad (and Nice fixes)

Problems of reduce

Contents
Motivation
Lifting
Problems
Problems of reduce
No End
Internal Structure

Where does our `reduce` fail?

Problems of reduce

Contents
Motivation
Lifting
Problems
Problems of reduce
No End
Internal Structure

Where does our `reduce` fail?

- ✖ Sequences with unknown end

Problems of reduce

Contents
Motivation
Lifting
Problems
Problems of reduce
No End
Internal Structure

Where does our `reduce` fail?

- ✖ Sequences with unknown end
- ✖ Sequences with internal structure (aka segmented)

Problems of reduce

Contents
Motivation
Lifting
Problems
Problems of reduce
No End
Internal Structure

Where does our `reduce` fail?

- ✖ Sequences with unknown end
- ✖ Sequences with internal structure (aka segmented)
- ✖ Sequentialism ($\neg$ parallelism) ...

Problems of reduce

Contents
Motivation
Lifting
Problems
Problems of reduce
No End
Internal Structure

Where does our `reduce` fail?

- ✖ Sequences with unknown end
- ✖ Sequences with internal structure (aka segmented)
- ✖ Sequentialism ($\neg$ parallelism) . . . *Different talk.*



(C) Alexandre Duret-Lutz, Flicker CC2.0

Let's count the e's

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

```
const char* text = read();
num['e'] = reduce(text, ???, 0, counter('e'));
```

What to put for ??? ???

1. Find the end by calling `strlen`
2. *somehow* make up an iterator for ???
3. Forget that *** generic reduce, roll our own `count_char`
4. *or something even more clever?*

Let's make an `end`

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

Can we make up an iterator value that *behaves* the right way?
Let's look at the non-generic code:

```
int count_char(char* b, char c)
{
  int res = 0;
  while(*b != 0) {
    ++res;
    ++b;
  }
  return res;
}
```

Let's make an `end`

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

Can we make up an iterator value that *behaves* the right way?
Compare to generic code:

```
T reduce(It b, It end, T init, Op op)
{
  int res = init;
  while(*b != end) {
    res = op(res, *b);
    ++b;
  }
  return res;
}
```

- ✖ So `b == end` must be equivalent to `*b == 0`
- ✖ Otherwise `b == b1` must have normal semantics

Let's make an `end`

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

- ✖ So `b == end` must be equivalent to `*b == 0`
- ✖ Otherwise `b == b1` must have normal semantics

So `end` must have a value *distinct* from all possible values:

```
class cstring_it {
  char *p;
  cstring_it()          : p(nullptr) {} // end
  cstring_it(char *p) : p(p)       {} // normal
}
```

Let's make an `end`

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

- So `b == end` must be equivalent to `*b == 0`
- Otherwise `b == b1` must have normal semantics

Also,

- `==` must be symmetric
- `end == end` must be true

```
bool operator==(cstring_it l, cstring_it r)
{
  if(l.p == 0) return r.p == 0 || *(r.p) == 0;
  if(r.p == 0) return l.p == 0 || *(l.p) == 0;
  return (r.p == l.p);
}
```

Using the dummy `end`

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

```
const char* text = read();
num['e'] = reduce(cstring_it(text), cstring_it(),
                  0, counter('e'));
```

Works just fine . . . All good?

Iterator category

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

Quiz: What's the (best achievable) iterator category of `cstring_it` ?

- ✖ Input iterator?
- ✖ Forward iterator?
- ✖ Bidirectional iterator?
- ✖ Random access iterator?

Iterator category

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

Quiz: What's the (best achievable) iterator category of `cstring_it` ?

- ✖ Input iterator?
- ✖ Forward iterator?
- ✖ Bidirectional iterator?
- ✖ Random access iterator?

Forward iterator. We can't do `--` on end.

Iterator category

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

Quiz: Is the generated code equivalent to the non-generic version?

```
int count_char(char* b, char c)
{
  int res = 0;
  while(*b != 0) {
    ++res;
    ++b;
  }
  return res;
}
```

Iterator category

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

The generated code is rather equivalent to this:

```
...
{
  int res = 0;
  while(!( (end.p == 0 && (b.p == 0 || *b == 0))
          ||(end.p != 0 && (b.p != 0 && (b.p == end.p))))
        )
{
    ++res;
    ++b;
  }
  return res;
}
```

Not good.

Problem of dummy `end`

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

What's the problem?

Problem of dummy `end`

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

What's the problem?

- ✖ We map a specific condition (end of string) to a specific *value*, and must then test for it.

Problem of dummy `end`

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

What's the problem?

- ✖ We map a specific condition (end of string) to a specific *value*, and must then test for it.
- ✖ We *know* that logic is not needed in the specific case . . . but comparison operator must work for all cases

Problem of dummy `end`

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

What's the problem?

- ✖ We map a specific condition (end of string) to a specific *value*, and must then test for it.
- ✖ We *know* that logic is not needed in the specific case . . . but comparison operator must work for all cases

Solution: Map *specific condition* into specific *type*!

A sentinel type for `end`

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

Solution: factor out specific condition into specific type!

```
class cstring_end {};
```

A sentinel type for `end`

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

Solution: factor out specific condition into specific type!

```
class cstring_end {};
```

Now comparison is easy:

A sentinel type for `end`

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

Solution: factor out specific condition into specific type!

```
class cstring_end {};
```

Now comparison is easy:

```
bool operator==(const char* lhs, cstring_end)
{ return (*lhs) == 0; }
```

A sentinel type for `end`

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

Solution: factor out specific condition into specific type!

```
class cstring_end {};
```

Now comparison is easy:

```
bool operator==(const char* lhs, cstring_end)
{ return (*lhs) == 0; }
```

But wait . . . does our reduce need a fix?

Reduce with sentinel type

Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

Let `reduce` accept a different *sentinel type* for `end` argument:

```
template<class Iter, class End, class T, class Op>
T reduce(Iter begin, End end, T init, Op op)
{
  T s = init;
  while(begin != end) {
    s = op(s,*begin);
    ++begin;
  }
  return s;
}
```

Reduce with sentinel type

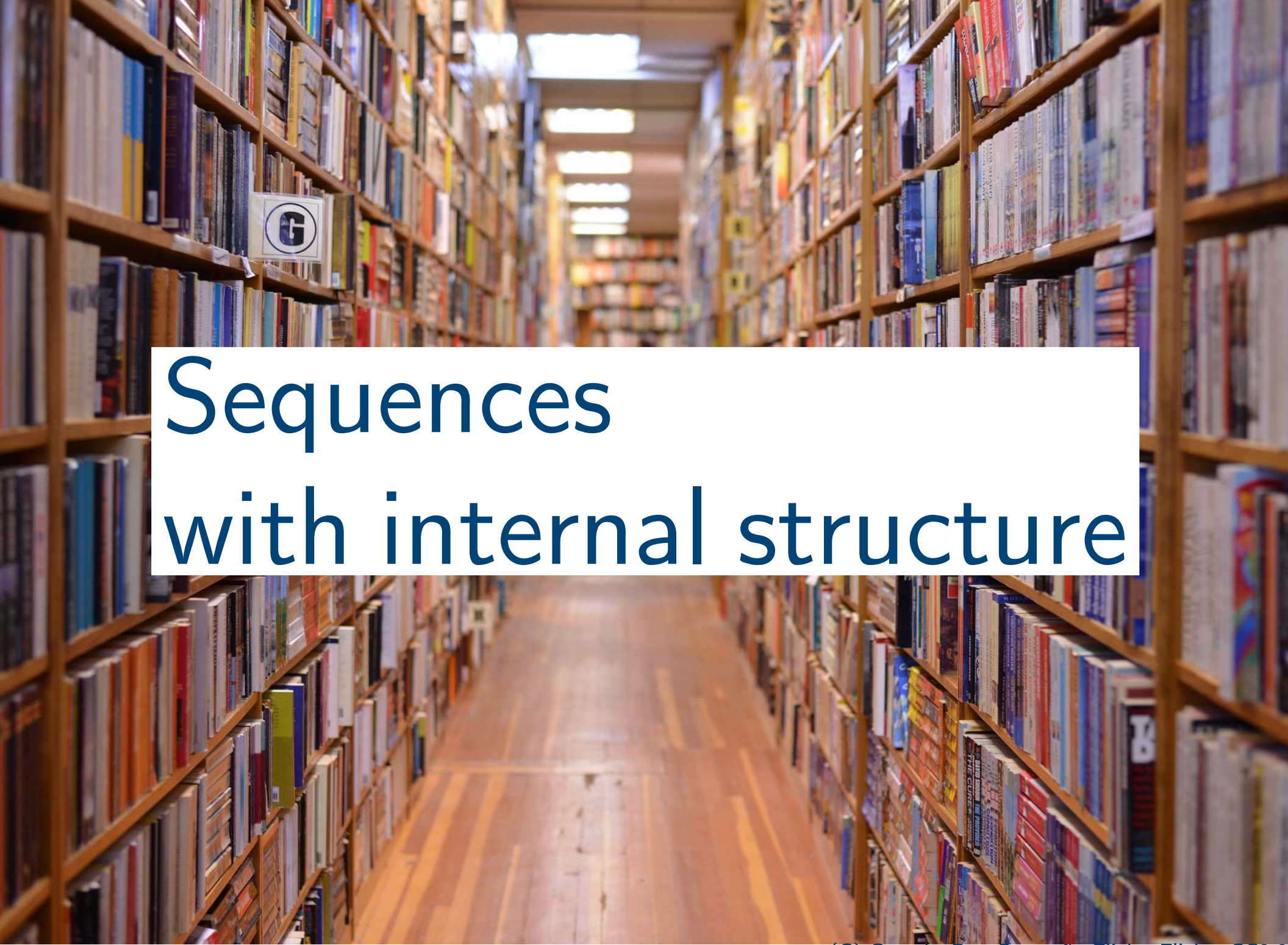
Contents
Motivation
Lifting
Problems
No End
Ex. C-string
Dummy end
Using dummy end
Category
Code gen
Analysis
A sentinel type
Reduce w. sentinel
Internal Structure

Let `reduce` accept a different *sentinel type* for `end` argument:

```
template<class Iter, class End, class T, class Op>
T reduce(Iter begin, End end, T init, Op op)
{
  T s = init;
  while(begin != end) {
    s = op(s,*begin);
    ++begin;
  }
  return s;
}
```

Still works for standard ranges (i.e. `begin`, `end` of same type)

References: [Niebler, Kuehl]



Sequences with internal structure

(C) Germán Poo-Caamaño Flickr, Flicker CC2.0

vector vs. deque

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

What performance difference do you expect?

```
int N = 1000*100;
vector<float> v(N);
deque <float> d(N);

// Benchmark 1
float resv = sum(v.begin(), v.end(), 0.0f);

// Benchmark 2
float resd = sum(d.begin(), d.end(), 0.0f);
```

Under the hood of deque

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

vector version ca. 4x faster
(g++ 4.9.2, Intel Core 2 w. SSE4.1)

Under the hood of deque

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

vector version ca. 4x faster
(g++ 4.9.2, Intel Core 2 w. SSE4.1)

Why?

Under the hood of deque

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

vector version ca. 4x faster
(g++ 4.9.2, Intel Core 2 w. SSE4.1)

Why?

A look into std::deque reveals:

- ✖ It is basically an array of pointers to fixed-sized arrays.
- ✖ Such a data structure is called *segmented* (nested, hierarchical)
- ✖ Data is not in contigous memory, but in disjoint chunks (segments or extents)

Segmented Data structures

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Examples:

- ✖ nested containers, `vector<vector<double>>`
- ✖ arrays of pointers to arrays `T**`
- ✖ DS optimized for multithreaded access (padding, NUMA-aware allocation, blocked data)

Segmented Data structures

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Examples:

- nested containers, `vector<vector<double>>`
- arrays of pointers to arrays `T**`
- DS optimized for multithreaded access (padding, NUMA-aware allocation, blocked data)

Questions:

- How to construct a flat iterator over nested sequences?
- How to exploit the hierarchical structure?

Segmented Data structures - Definitions

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

- Segmented sequence = sequence of *segments*
  - → major sequence (MS)

Segmented Data structures - Definitions

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

- Segmented sequence = sequence of *segments*
  - ➔ major sequence (MS)
- Segments themselves represent sequences
  - ➔ minor sequences (ms), flat or segmented

Segmented Data structures - Definitions

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

- Segmented sequence = sequence of *segments*
  - ➔ major sequence (MS)
- Segments themselves represent sequences
  - ➔ minor sequences (ms), flat or segmented
- Whole sequence = Logical concatenation of minor sequences
  - ➔ also flattened sequence

Segmented Data structures - Definitions

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

- Segmented sequence = sequence of *segments*
  - ➔ major sequence (MS)
- Segments themselves represent sequences
  - ➔ minor sequences (ms), flat or segmented
- Whole sequence = Logical concatenation of minor sequences
  - ➔ also flattened sequence
- Major iterator = Iterator over the top-level segments
- Minor iterator = Iterator over the sequence of one segment

Segmented Data structures - Definitions

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

- Segmented sequence = sequence of *segments*
  - ➔ major sequence (MS)
- Segments themselves represent sequences
  - ➔ minor sequences (ms), flat or segmented
- Whole sequence = Logical concatenation of minor sequences
  - ➔ also flattened sequence
- Major iterator = Iterator over the top-level segments
- Minor iterator = Iterator over the sequence of one segment
- Flattening Iterator = Iterator over the whole sequence
  - ➔ basically a pair (M,m) of major + minor iterator

Outline of a flattening iterator (pseudocode)

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

```
class flattening_it {
   Major M;
   Minor m;

   void increment() {
     ++m;
     if(m == end(M)) {
       ++M;
       m = begin(M);
     }
   }

   bool equal(Major rhs) {
    return M == rhs.M && m == rhs.m;
   }

   value_type deref() { return *m; }
};
```

Flattening Iterator: How to say end?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

How to represent the end iterator E_W of the whole sequence?

Flattening Iterator: How to say end?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

How to represent the end iterator E_W of the whole sequence?

Let E = end of the major sequence, and PE = prev(E)

Flattening Iterator: How to say end?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

How to represent the end iterator E_W of the whole sequence?

Let E = end of the major sequence, and PE = prev(E)

1. E_W = (PE, some valid pos(PE)) (deque approach)

Flattening Iterator: How to say end?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

How to represent the end iterator E_W of the whole sequence?

Let E = end of the major sequence, and PE = prev(E)

1. E_W = (PE, some valid pos(PE)) (deque approach)
2. E_W = (PE, end(PE))

Flattening Iterator: How to say end?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

How to represent the end iterator E_W of the whole sequence?

Let E = end of the major sequence, and PE = prev(E)

1. E_W = (PE, some valid pos(PE)) (deque approach)
2. E_W = (PE, end(PE))
3. E_W = (E, .) (value of minor does not matter)

Flattening Iterator: How to say end?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

How to represent the end iterator E_W of the whole sequence?

Let E = end of the major sequence, and PE = prev(E)

1. E_W = (PE, some valid pos(PE)) (deque approach)
2. E_W = (PE, end(PE))
3. E_W = (E, .) (value of minor does not matter)

The choice will affect

Flattening Iterator: How to say end?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

How to represent the end iterator E_W of the whole sequence?

Let E = end of the major sequence, and PE = prev(E)

1. E_W = (PE, some valid pos(PE)) (deque approach)
2. E_W = (PE, end(PE))
3. E_W = (E, .) (value of minor does not matter)

The choice will affect

- ✖ Increment
- ✖ Comparison
- ✖ Initialization

Flattening Iterator: How to say end?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

How to represent the end iterator E_W of the whole sequence?

Let E = end of the major sequence, and PE = prev(E)

1. E_W = (PE, some valid pos(PE)) (deque approach)
2. E_W = (PE, end(PE))
3. E_W = (E, .) (value of minor does not matter)

The choice will affect

- ✖ Increment
- ✖ Comparison
- ✖ Initialization

We choose option 3: E_W = (E, .) for generality.

Flattening iterator: End = (E,.)

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

```
template<class Major, class Minor>
class flattening_it {
   Major M, E;
   Minor m;
   typedef ??? iterator_category;

   flattening_it(Major M, Major E, Minor m)
     : M(M), E(E), m(m) {}

   void increment() {
     ++m;
     if(m == end(M)) {
       ++M;
       if(M != E)
           m = begin(M);
     }
   }
...  <continued on next slide>
```

Flattening iterator: End = (E,.) (ctn'd)

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

```
template<class Major, class Minor>
class flattening_it {
   Major M, E;
   Minor m;

   ...

   bool equal(rhs) {
     return  (M == rhs.M)
          && ((M == E) || (m == rhs.m));
   }

   value_type deref() {
     return *m;
   }
};
```

Flattening Iterator: Issues accessing minor sequences

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

How to access the minor sequences?

Flattening Iterator: Issues accessing minor sequences

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

How to access the minor sequences?

- ✖ `vector<vector<T>>` : `begin(*M)`, `end(*M)`

Flattening Iterator: Issues accessing minor sequences

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

How to access the minor sequences?

- ✖ `vector<vector<T>>` : `begin(*M)`, `end(*M)`
- ✖ `T **` : `*M` is begin, `*M` + N is end,

  ➔ but how to get size N of sequence starting at `*M` ??

Flattening Iterator: Issues accessing minor sequences

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

How to access the minor sequences?

- ✖ `vector<vector<T>>` : `begin(*M)`, `end(*M)`
- ✖ `T **` : `*M` is begin, `*M` + N is end,

  ➔ but how to get size N of sequence starting at `*M` ??

Need extra template parameter encapsulating these details.
Traits *not* sufficient: e. g. `T**` doesn't carry enough information.

Not elaborated further in this talk.

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

Category of `flattening_it<Major,Minor>`:

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Category of `flattening_it<Major,Minor>`:
At most the weaker of category(Major), category(Minor)

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

Category of `flattening_it<Major,Minor>`:
At most the weaker of category(Major), category(Minor)

Consider `vector<vector<int>>`.
Can corresponding flattening iterator be Random Access?

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Category of `flattening_it<Major,Minor>`:
At most the weaker of category(Major), category(Minor)

Consider `vector<vector<int>>`.
Can corresponding flattening iterator be Random Access?

Can we compute in $O(1)$

1. `it + n` ?

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Category of `flattening_it<Major,Minor>`:
At most the weaker of category(Major), category(Minor)

Consider `vector<vector<int>>`.
Can corresponding flattening iterator be Random Access?

Can we compute in $O(1)$

1. `it + n` ?
2. `i1 - i2` ?

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Category of `flattening_it<Major,Minor>`:
At most the weaker of category(Major), category(Minor)

Consider `vector<vector<int>>`.
Can corresponding flattening iterator be Random Access?

Can we compute in $O(1)$

1. `it + n` ?
2. `i1 - i2` ?
3. `i1 < i2` ?

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Category of `flattening_it<Major,Minor>`:
At most the weaker of category(Major), category(Minor)

Consider `vector<vector<int>>`.
Can corresponding flattening iterator be Random Access?

Can we compute in $O(1)$

1. `it + n` ?
2. `i1 - i2` ?
3. `i1 < i2` ?

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

Can we compute in $O(1)$?

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

Can we compute in $O(1)$?

1. `it + n` : $O(S)$ (where S = size(MS) is no. of segments)

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Can we compute in $O(1)$?

1. `it + n` : $O(S)$ (where S = size(MS) is no. of segments)
   - ✖ Fixed ms sizes: $O(1)$ (deque)

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Can we compute in $O(1)$?

1. `it + n` : $O(S)$ (where S = size(MS) is no. of segments)
   - ✖ Fixed ms sizes: $O(1)$ (deque)
   - ✖ With prefix sum of ms sizes: $O(\log(S))$

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Can we compute in $O(1)$?

1. `it + n` : $O(S)$ (where S = size(MS) is no. of segments)
   - ✖ Fixed ms sizes: $O(1)$ (deque)
   - ✖ With prefix sum of ms sizes: $O(\log(S))$
2. `i1 - i2` : $O(S)$

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Can we compute in $O(1)$?

1. `it + n` : $O(S)$ (where S = size(MS) is no. of segments)
   - ✖ Fixed ms sizes: $O(1)$ (deque)
   - ✖ With prefix sum of ms sizes: $O(\log(S))$
2. `i1 - i2` : $O(S)$
   - ✖ Prefix sum of segment sizes / fixed sizes: $O(1)$

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Can we compute in $O(1)$?

1. `it + n` : $O(S)$ (where S = size(MS) is no. of segments)
   - ✖ Fixed ms sizes: $O(1)$ (deque)
   - ✖ With prefix sum of ms sizes: $O(\log(S))$
2. `i1 - i2` : $O(S)$
   - ✖ Prefix sum of segment sizes / fixed sizes: $O(1)$
3. `i1 < i2` : $O(1)$

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Can we compute in $O(1)$?

1. `it + n` : $O(S)$ (where S = size(MS) is no. of segments)
   - ✖ Fixed ms sizes: $O(1)$ (deque)
   - ✖ With prefix sum of ms sizes: $O(\log(S))$
2. `i1 - i2` : $O(S)$
   - ✖ Prefix sum of segment sizes / fixed sizes: $O(1)$
3. `i1 < i2` : $O(1)$

Thus, in general the iterator category is bidirectional at most. (Except fixed size segments).

Flattening Iterator: Which iterator category?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Can we compute in $O(1)$?

1. `it + n` : $O(S)$ (where S = size(MS) is no. of segments)
   - ✖ Fixed ms sizes: $O(1)$ (deque)
   - ✖ With prefix sum of ms sizes: $O(\log(S))$
2. `i1 - i2` : $O(S)$
   - ✖ Prefix sum of segment sizes / fixed sizes: $O(1)$
3. `i1 < i2` : $O(1)$

Thus, in general the iterator category is bidirectional at most.
(Except fixed size segments).
However:

- ✖ `i < j` can be computed in $O(1)$
- ✖ distance and advance *can* be considerably faster than $O(n)$ (depending on relative sizes of ms and MS.)

Issues with Flattening: Generic code

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

Remember: reduce on vector 4x faster than on deque.

How does the generated code look like?

```
template<class It, class T, class Op>
T reduce(It b, It e, T init, Op op)
{
   T res = init;
   while(b != e) {
     res = op(res, *b);
     ++b;
   }
}
```

With a flattening iterator, transforms to ...

Issues with Flattening: Generated code

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

```
template<class It, class T, class Op>
T reduce(It b, It e, T init, Op op)
{
   T res = init;
   while(! (b.M == b.E && b.M == e.M)
          ||(b.M != b.E && b.m == e.m))
   {
     res = op(res, *b);
     ++b.m;
     if(b.m == end(*b.M) {
       ++(b.M);
       if(b.M != b.E)
        b.m = begin(*b.M);
     }
   }
}
```

Issues with Flattening: What's wrong?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

What is the issue here?

Issues with Flattening: What's wrong?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

What is the issue here?

✖ Some performance loss because of extra logic . . .

Issues with Flattening: What's wrong?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

What is the issue here?

- ✖ Some performance loss because of extra logic . . .
- ✖ Worse: **Unlikely to vectorize!**

Issues with Flattening: What's wrong?

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

What is the issue here?

- ✖ Some performance loss because of extra logic . . .
- ✖ Worse: **Unlikely to vectorize!**

What we *really* want, is . . .

Issues with Flattening

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

```
template<class It, class T, class Op>
T reduce(It b, It e, T init, Op op)
{
  T res = init;
  while(b.M != e.M) {
    for(It::minor m = begin(b.M); m != end(b.M); ++m)
      res = op(res, *m);
    ++b.M;
  }
}
```

- ✖ Exploits DS structure.
- ✖ Has better chance to vectorize.
- ✖ No degradation of iterator categories.

Note: Example is simplified. Why?

Getting the structure back

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

How can we achieve this structure-aware code?

Getting the structure back

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

How can we achieve this structure-aware code?

- ✖ Iterator type encodes (most of) data structuring

Getting the structure back

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

How can we achieve this structure-aware code?

- ✖ Iterator type encodes (most of) data structuring
- ✖ Must retrieve structure from the iterator

Getting the structure back

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

How can we achieve this structure-aware code?

- ✖ Iterator type encodes (most of) data structuring
- ✖ Must retrieve structure from the iterator
- ✖ Use that to implement structure-aware algorithms

Getting the structure back

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

How can we achieve this structure-aware code?

- ✖ Iterator type encodes (most of) data structuring
- ✖ Must retrieve structure from the iterator
- ✖ Use that to implement structure-aware algorithms

From *flattening* to *structure-aware iterator!**

* also know as *segmented iterator*

Structure-aware (segmented) iterators

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

What do we need from a segmented iterator?

Structure-aware (segmented) iterators

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

What do we need from a segmented iterator?

- ✖ Types for major and minor iterators
- ✖ Access to current major iterator
- ✖ Producing bounds begin & end of minor sequences from major iterator
- ✖ *end major, current minor iterator*

Structure-aware (segmented) iterators

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

What do we need from a segmented iterator?

- ✖ Types for major and minor iterators
- ✖ Access to current major iterator
- ✖ Producing bounds begin & end of minor sequences from major iterator
- ✖ *end major, current minor iterator*

How to?

1. extending a flattening iterator type
2. traits techniques (non-invasive)

Structure-aware (segmented) iterators

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

What do we need from a segmented iterator?

- ✖ Types for major and minor iterators
- ✖ Access to current major iterator
- ✖ Producing bounds begin & end of minor sequences from major iterator
- ✖ *end major, current minor iterator*

How to?

1. extending a flattening iterator type
2. traits techniques (non-invasive)

Consider:

- ✖ Flattening iterator must be a class already (even for `T**`)
- ✖ Functionality must be already present in flattening iterator
- ✖ (but may not be accessible (cf. deque iterator))

Structure-aware (segmented) iterators

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Extending `flattening_iterator`:

```
template<class Major, class Minor>
class segmented_iterator {
public:
   typedef Major major_iterator;
   typedef Minor minor_iterator;

   major_iterator curr_major();
   major_iterator end_major();

   minor_iterator begin(Major MI);
   minor_iterator end  (Major MI);
   minor_iterator curr_minor();

   // + flattening_iterator stuff
};
```

Structure-aware algorithms: reduce

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

A general structure-aware accumulate must handle the following parts of the whole sequences:

Structure-aware algorithms: reduce

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

A general structure-aware accumulate must handle the following parts of the whole sequences:

1. Run reduce on first, possibly incomplete ms (`m > begin(M)`)
2. Run reduce on complete ms
3. Run reduce on last, possibly incomplete ms (`m < end(M)`)

Structure-aware algorithms: reduce

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

A general structure-aware accumulate must handle the following parts of the whole sequences:

1. Run reduce on first, possibly incomplete ms (`m > begin(M)`)
2. Run reduce on complete ms
3. Run reduce on last, possibly incomplete ms (`m < end(M)`)

Maintain common interface for flat & structure-aware iterators:

Structure-aware algorithms: reduce

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

A general structure-aware accumulate must handle the following parts of the whole sequences:

1. Run reduce on first, possibly incomplete ms (`m > begin(M)`)
2. Run reduce on complete ms
3. Run reduce on last, possibly incomplete ms (`m < end(M)`)

Maintain common interface for flat & structure-aware iterators:

- ✖ define a type map for the structure-awareness property
- ✖ top-level reduce branches to flat and hierarchical versions

Structure-aware algorithms: reduce

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

A general structure-aware accumulate must handle the following parts of the whole sequences:

1. Run reduce on first, possibly incomplete ms (`m > begin(M)`)
2. Run reduce on complete ms
3. Run reduce on last, possibly incomplete ms (`m < end(M)`)

Maintain common interface for flat & structure-aware iterators:

- ✖ define a type map for the structure-awareness property
- ✖ top-level reduce branches to flat and hierarchical versions
  - ➔ hierarchical version calls again top-level reduce to match structure-aware minor iterators

Structure-aware reduce: Flat / hierachical branch

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

```
template<class It, class T, class Op>
T reduce(It b, It e, T init, Op op)
{
  return reduce(b,e,init,op,
                typename is_hier<It>::value());
}

template<class It, class T, class Op>
T reduce(It b, It e, T init, Op op, hier_tag)
{ return reduce_hier(b,e,init,op); }

template<class It, class T, class Op>
T reduce(It b, It e, T init, Op op, flat_tag)
{ return reduce_flat(b,e,init,op); }
```

Where our original reduce is renamed to `reduce_flat`.

Structure-aware reduce, part 1

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

```
template<class It, class T, class Op>
T reduce_hier(It hb, It he, T init, Op op)
{
  typedef typename It::major_iterator major;
  typedef typename It::minor_iterator minor;

  major B  = hb.curr_major(), E = he.curr_major();
  major ES = hb.end_major();
  minor b,e;
  // do first segment
  if(B == ES)
    return init;
  b = B.curr_minor();
  e = (B == E ? E.curr_minor() : hb.end(B));
  init = reduce(b,e,init,op);

  // do full segments ...  <continued>
```

Structure-aware reduce, part 2

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

```
  // do full segments
  if(B == E) // no segments left
    return init;
  ++B;
  while(B != E) {
    b = hb.begin(B);
    e = hb.end(B);
    init = reduce(b,e,init,op);
    ++B;
  }

  // do last segment: B == E
  if(B != ES) { // a segment is left
    b = hb.begin(B);
    e = he.curr_minor();
    init = reduce(b,e,init,op);
  }
  return init;
}
```

Structure-aware reduce: The Gain

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

Now lets look on the performance of hierarchical reduce:

Data structure	reduce	time (128)
`float *`	flat	1.0
`vector<float>`	flat	1.0
`deque<float>`	flat	4.1
`vector<vector<float>>`	flat	4.1
`vector<vector<float>>`	hier	1.6

After times, size of segments is given.
128 corresponds to deque value (g++ 4.9.2)

Structure-aware reduce: The Gain

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

Now lets look on the performance of hierarchical reduce:

Data structure	reduce	time (128)	time (1024)
`float *`	flat	1.0	1.0
`vector<float>`	flat	1.0	1.0
`deque<float>`	flat	4.1	4.1
`vector<vector<float>>`	flat	4.1	4.0
`vector<vector<float>>`	hier	1.6	1.0

After times, size of segments is given.
128 corresponds to deque value (g++ 4.9.2)

Generalizing the hierarchical approach

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

Implementing this for all (suitable) algorithms: A lot of work?

Generalizing the hierarchical approach

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Implementing this for all (suitable) algorithms: A lot of work?

Can we abstract from the concrete algorithm (reduce)?

Generalizing: Some (STL) algorithms

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

`all_of`	`for_each`	`count_if`	`mismatch`
`equal`	`find`	`adjacent_find`	`search`
`copy`	`fill`	`transform`	`replace`
`remove`	`replace`	`rotate`	`shuffle`
`partition`	`is_sorted`	`sort`	`merge`
`lower_bound`	`includes`	`set_union`	`make_heap`
`max_element`	`iota`	`accumulate`	`partial_sum`

...

Generalizing: Some (STL) algorithms

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

`all_of`	`for_each`	`count_if`	`mismatch`
`equal`	`find`	`adjacent_find`	`search`
`copy`	`fill`	`transform`	`replace`
`remove`	`replace`	`rotate`	`shuffle`
`partition`	`is_sorted`	`sort`	`merge`
`lower_bound`	`includes`	`set_union`	`make_heap`
`max_element`	`iota`	`accumulate`	`partial_sum`

...

Can we group algorithms having same "hierarchical pattern" ?

Generalizing: Groups of algorithms

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

Generalizing: Groups of algorithms

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

1. *reduce-type*: `count`, `accumulate`, `max_element`, ...

Generalizing: Groups of algorithms

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

1. *reduce-type*: `count`, `accumulate`, `max_element`, ...
2. *transform-type*: `fill`, `foreach`, `transform`, ...

Generalizing: Groups of algorithms

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

1. *reduce-type*: `count`, `accumulate`, `max_element`, ...
2. *transform-type*: `fill`, `foreach`, `transform`, ...
3. *find-type*: `all_of`, `find`, `is_`*xxx*, ...

Generalizing: Groups of algorithms

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

1. *reduce-type*: `count`, `accumulate`, `max_element`, ...
2. *transform-type*: `fill`, `foreach`, `transform`, ...
3. *find-type*: `all_of`, `find`, `is_`*xxx*, ...
4. *swap-type*: `rotate`, `remove`, `sort`, ...
5. ...

Generalizing: Groups of algorithms

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

1. *reduce-type*: `count`, `accumulate`, `max_element`, ...
2. *transform-type*: `fill`, `foreach`, `transform`, ...
3. *find-type*: `all_of`, `find`, `is_`*xxx*, ...
4. *swap-type*: `rotate`, `remove`, `sort`, ...
5. ...

We also must distinguish

- ✖ algorithms taking 2 or more ranges
- ✖ algorithms with predicates involving subsequences

..... leaving them out for now.

Algorithms of reduce-type

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

What is the common property of reduce-type algorithm f ?

Algorithms of reduce-type

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

What is the common property of reduce-type algorithm f ?

Common interface (modulo ordering and suppression)
r = f(r_0, R, other args)
where

1. $R = [b, e)$ is an input range
2. r_0 is an (optional) init value
3. Return value depends on R and $r0$ (+ optional other args)

Algorithms of reduce-type

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

What is the common property of reduce-type algorithm f ?

Common interface (modulo ordering and suppression)
r = f(r_0, R, other args)
where

1. $R = [b, e)$ is an input range
2. r_0 is an (optional) init value
3. Return value depends on R and $r0$ (+ optional other args)

The algorithms are *composable*:
Let $R = (R_1, R_2) = [b, m), [m, e)$. Then
f(r_0, R, args) = f(f(r_0,R_1, args), R_2, args)

Algorithms of reduce-type

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

Not all fit strictly, but can be adapted:

- ✖ `std::accumulate(b,e,init,op)` into `wrap_accumulate(init,b,e,op)`

Algorithms of reduce-type

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

Not all fit strictly, but can be adapted:

- ✖ `std::accumulate(b,e,init,op)` into `wrap_accumulate(init,b,e,op)`
- ✖ `std::count_if(b,e,pred)` into `wrap_count_if(init,b,e,pred)`

Algorithms of reduce-type

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

Not all fit strictly, but can be adapted:

- ✖ `std::accumulate(b,e,init,op)` into `wrap_accumulate(init,b,e,op)`
- ✖ `std::count_if(b,e,pred)` into `wrap_count_if(init,b,e,pred)`

We can even make fit transform-type algorithms:

Algorithms of reduce-type

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Not all fit strictly, but can be adapted:

- ✖ `std::accumulate(b,e,init,op)` into `wrap_accumulate(init,b,e,op)`
- ✖ `std::count_if(b,e,pred)` into `wrap_count_if(init,b,e,pred)`

We can even make fit transform-type algorithms:

- ✖ `std::fill(b,e,val)` into `wrap_fill(dummy,b,e,val)`

Wrapping algorithms

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap up

Example: wrapping `std::count_if`:

```
class wrap_count_if {
public:
   template<class T, class It, class Pred>
   T
   f(T init, It b, It e, Pred p)
   { return init + std::count_if(b,e,p); }
};
```

Now we are ready for the generic hierarchical algorithm . . .

A generic hierarchical algorithm

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

The basic logic is identical to hierarchical reduce:

- ✖ Top-level `reduction_algo` branches : iterator flat or hierarchical?
- ✖ `reduction_algo_flat` : usual flat algorithm
- ✖ `reduction_algo_hier` : hierarchical version

A generic hierarchical algorithm

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

The basic logic is identical to hierarchical reduce:

- ✖ Top-level `reduction_algo` branches : iterator flat or hierarchical?
- ✖ `reduction_algo_flat` : usual flat algorithm
- ✖ `reduction_algo_hier` : hierarchical version

What arguments does `reduction_algo` take?

A generic hierarchical algorithm

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

The basic logic is identical to hierarchical reduce:

- ✖ Top-level `reduction_algo` branches : iterator flat or hierarchical?
- ✖ `reduction_algo_flat` : usual flat algorithm
- ✖ `reduction_algo_hier` : hierarchical version

What arguments does `reduction_algo` take?

- ✖ r_0 (as `init`) and R (as `b`, `e`)

A generic hierarchical algorithm

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

The basic logic is identical to hierarchical reduce:

- Top-level `reduction_algo` branches : iterator flat or hierarchical?
- `reduction_algo_flat` : usual flat algorithm
- `reduction_algo_hier` : hierarchical version

What arguments does `reduction_algo` take?

- r_0 (as `init`) and $\boldsymbol{R}$ (as `b`, `e`)
- the algorithm $\boldsymbol{f}$

A generic hierarchical algorithm

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

The basic logic is identical to hierarchical reduce:

- Top-level `reduction_algo` branches : iterator flat or hierarchical?
- `reduction_algo_flat` : usual flat algorithm
- `reduction_algo_hier` : hierarchical version

What arguments does `reduction_algo` take?

- r_0 (as `init`) and R (as `b`, `e`)
- the algorithm f
- *any number of additional arguments*

A generic hierarchical algorithm

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

```
template<typename Algo, typename T, typename It,
         typename ...Other>
T reduction_algo(Algo f, T init, It b, It e,
                 Other... o);

template<typename Algo, typename T, typename It,
         typename ...Other>
T reduction_algo_flat(Algo f, T init, It b, It e,
                      Other... o)
{ return f(init,b,e,o...); }

template<typename Algo, typename T, typename It,
         typename ...Other>
T reduction_algo_hier(Algo f, T init, It b, It e,
                      Other... o)
{ /* hierarchical implementation as per reduce */ }
```

Using the generic hierarchical algorithm

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Time to use our generic hierarchical implementation!

```
template<class It, class Pred>
typename iterator_traits<It>::difference_type
count_if_hier(It b, It e, Pred p)
{ reduction_algo(wrap_count_if(), 0, b,e,p); }
```

Using the generic hierarchical algorithm

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

Time to use our generic hierarchical implementation!

```
template<class It, class Pred>
typename iterator_traits<It>::difference_type
count_if_hier(It b, It e, Pred p)
{ reduction_algo(wrap_count_if(), 0, b,e,p); }
```

```
template<class It, class T, class Pred>
T accumulate_hier(It b, It e, T init, Pred p)
{
 return reduction_algo(wrap_accumulate(), init,b,e,p);
}
```

Hierarchical algorithms: Wrap-up

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up

- Hierarchical version: high potential gain (for vectorizable algorithms)
- Implementable with few high-level generic versions (reduction-type, find-type, . . .)
- There is some space between bidirectional and random access!

Further remarks & steps to go:

- Complete classification of algorithms
- . . . may find hints for improving interfaces!
- Develop high-level strategies for other classes of algorithms
- *Similar strategy applicable to parallelization!?*

конец

References and Further Reading

Contents
Motivation
Lifting
Problems
No End
Internal Structure
A benchmark
Deque
Segmented DS
Flattening
Sample Impl
Minor seq. access
Iterator category
Performance flat
Segmented Iter
Hierarchical reduce
Performance hier.
Generic hierarchical
Generic implementation
Using generic impl
Wrap-up
References

- Eric Niebler, Range Concepts, Part 1 of 4: Delimited Ranges
- Dietmar Kuehl, STL 2.0 (kuhllib)
- Matthew Austern, Segmented Iterators and Hierarchical Algorithms, Dagstuhl Seminar on Generic Programming, 1998.
- Holger Stengel, C++ programming techniques for High Performance Computing on syst